AN SE-RESNET-26 UNDER 5M PARAMETERS FOR CIFAR-10: WHAT HELPED, WHAT DID NOT, AND WHY

PROJECT REPORT

Uvindu Salgado
NetID uis202
uis202@nyu.edu

Michael Wang
NetID mzw2010
mzw2010@nyu.edu

November 2026

Kaggle team: Punters · **Code:** https://github.com/breeze215/cifar10-resnet-5m

ABSTRACT

We design and train a residual network with at most 5 million parameters for the course's CIFAR-10 Kaggle challenge, whose hidden test set is drawn from a different distribution than the training images. Our final model, an SE-ResNet-26 with 4,810,754 trainable parameters trained from scratch, reaches 97.9% accuracy on a held-out validation split and **89.7%** on the public leaderboard (private: **TBD**). We ran thirteen training runs (ten distinct experiments) and report all of them, including the ones that failed. The largest leaderboard gain (+3.0 points) came from stronger data augmentation. After that, further validation gains of up to +0.5 points, sharpness-aware minimization, knowledge distillation and augmentation aimed at robustness all left the public score between 89.0% and 89.9%. We trace this to two causes: the test images are brighter, higher in contrast and more often grayscale than the training images, so in-distribution gains transfer only partially; and a 1,000-image leaderboard has a standard error of about one point, larger than most of the differences we were chasing.

1 Overview

The task is to classify 32×32 CIFAR-10 images [Krizhevsky, 2009] with a convolutional network containing residual connections, using at most 5M trainable parameters, no pretrained weights and no external data. Models are trained on the 50,000 labeled training images and scored on a hidden set of 2,000 images, split into a public half (shown during the competition) and a private half (used for the final ranking).

Summary of findings.

- **Final model.** SE-ResNet-26: three stages of 5/4/3 BasicBlocks at 64/128/256 channels with squeeze-and-excitation gates [Hu et al., 2018], 4,810,754 parameters, trained for 600 epochs with TrivialAugment [Müller and Hutter, 2021], CutMix [Yun et al., 2019], label smoothing, an exponential moving average of the weights and test-time augmentation. Validation accuracy 97.90%; public leaderboard 89.7%.
- **What moved the leaderboard.** Replacing a basic crop/flip/Cutout pipeline with TrivialAugment, CutMix and weight averaging (together with a deeper network) raised the public score from 86.9% to 89.9%.
- **What did not.** Training on all 50k images, SAM [Foret et al., 2021], AugMix [Hendrycks et al., 2020], distillation [Hinton et al., 2015], the deeper SE network (+0.5 points on validation) and robustness-oriented augmentation (+0.6 points on shifted validation images) all scored between 89.0% and 89.8% on the public leaderboard.
- **Why.** The hidden test set is shifted relative to the training data (Section 3.1), and the public score is too noisy to resolve sub-point differences (Section 3.2). We therefore chose our two final submissions by validation and robustness evidence, not by public score.

2 Methodology

2.1 Evaluation protocol

Every validation number in this report uses the same 5,000 images, held out from the training set with a fixed seed (seed 0); these images are never used for training, early stopping or tuning other than as reported. Validation accuracy is computed with the EMA weights and averaged over the image and its horizontal flip. For submissions we retrain the chosen recipe on all 50,000 images (a "full" run, which has no validation split) and predict with 10 views (2 flips $\times$ 5 shifts of ± 2 pixels). Training ran on Kaggle NVIDIA T4 GPUs in mixed precision [Micikevicius et al., 2018] with PyTorch [Paszke et al., 2019]; we usually ran two jobs at once, one per GPU.

2.2 Architecture

A residual block computes $\mathrm{ReLU}(S(x) + F(x))$, where F is conv3×3–BN–ReLU–conv3×3–BN [He et al., 2016, Ioffe and Szegedy, 2015] and S is the identity, or a 1×1 convolution with BN when the shape changes. Following the usual CIFAR convention (as in Liu, 2017), the stem is a single 3×3, stride-1 convolution with no max-pooling, so the first stage works at full 32×32 resolution.

Where the parameter budget goes. The budget rules out the standard four-stage ResNet-18 widths: a single block at 512 channels costs 3.67M parameters. We therefore use three stages (resolutions 32, 16 and 8) and spend the budget on depth. Table 1 lists the three architectures we trained. v1 used fewer, wider blocks; v2 went deeper at standard widths, the same layout as a previous project in this course [Thakur et al., 2023]; the final network adds one block to the first stage and an SE gate to every block. SE costs only 39,608 parameters (0.8% of the total), and adding one more block to stage 2 instead would have exceeded the budget (5,033,282).

Table 1: Architectures we trained. Layer counts follow the ResNet-18 convention (main-path convolutions plus the classifier). MACs: multiply-accumulates per 32×32 image.

Model	Widths	Blocks per stage	Layers	Params	MACs
v1 ResNet-14	80 / 160 / 320	2 / 2 / 2	14	4,337,370	658M
v2 ResNet-24	64 / 128 / 256	4 / 4 / 3	24	4,697,162	799M
Final: SE-ResNet-26	64 / 128 / 256	5 / 4 / 3 + SE	26	**4,810,754**	874M
WRN-28-10 (teacher, not trained)	160 / 320 / 640	4 / 4 / 4	28	36,479,194	5,243M

2.3 Training recipe

All runs use SGD with Nesterov momentum 0.9 [Sutskever et al., 2013], learning rate 0.1 with a 5-epoch linear warm-up and cosine decay [Loshchilov and Hutter, 2017], weight decay 5×10^{-4} (not applied to BN and bias parameters), batch size 128, and label smoothing 0.1 [Szegedy et al., 2016]. v1 used reflect-padded random crops, horizontal flips and Cutout [DeVries and Taylor, 2017]. From v2 on we add TrivialAugmentWide (one random photometric or geometric operation per image), RandomGrayscale ($p = 0.05$), CutMix on half of the batches, and an exponential moving average (EMA) of the weights with decay 0.9995 [Polyak and Juditsky, 1992], which we use for all evaluation. The final model trains for 600 epochs (about 32 s per epoch on a T4).

2.4 The sequence of experiments

Table 2 lists every run in the order we ran them. Each step was a response to the previous result; Section 3 explains the failures.

3 What went wrong, and why

3.1 Validation accuracy stopped predicting the leaderboard

Going from v1 to v2 added 0.74 points of validation accuracy and 3.0 points on the public leaderboard. Going from v2 to v4 added another 0.50 points of validation accuracy and *lost* 0.2 points on the public leaderboard (Figure 1). The 8-point gap between validation (97–98%) and test (89–90%) accuracy is much larger than the validation error itself, so the test images are very unlikely to come from the training distribution.

Table 2: All runs. Val: EMA accuracy on our 5k split (flip-averaged). Runs marked "50k" train on all images and have no validation split; their val column shows the matching 45k run. Public: Kaggle public leaderboard (1,000 images). †Both v3 runs were uploaded from one notebook version and scored 89.0 and 89.3; we did not record which was which.

#	Run	Change from previous	Epochs	Val (%)	Public (%)
1	v1	ResNet-14; crop, flip, Cutout, label smoothing	200	96.66	86.9
2	v2	ResNet-24; TrivialAugment, grayscale, CutMix, EMA, TTA	300	97.40	**89.9**
3	v3 full	v2 recipe on all 50k images	300	(97.40)	89.0 / 89.3†
4	v3 SAM	v2 + SAM ($\rho = 0.05$)	200	97.22	89.3 / 89.0†
5	AugMix (MW)	v2 with AugMix instead of TrivialAugment	300	97.20	89.4
6	v2 50k (MW)	v2 on 50k, RandomGrayscale removed	300	(97.40)	89.8
7	distillation	v2 student; teachers v2 + SAM (50k run submitted)	200	97.34	89.0
8	**v4** (`final`)	SE-ResNet-26, 600 epochs (50k run submitted)	600	**97.90**	89.7
9	**robust** (`final^2`)	v4 + AugMix|TrivialAugment, jitter, blur	600	97.82	89.6
10	hard-view fine-tune	continue v4 on hardest-of-two views	+200	97.44	—

We compared aggregate pixel statistics of the training images with those of the (unlabeled) test images (Table 3). The test images are brighter, higher in contrast, sharper, and three times as likely to be grayscale. Together with the size of the gap, this resembles re-collected test sets such as CIFAR-10.1 [Recht et al., 2018, 2019], on which models typically lose several points while keeping their ranking, the "accuracy on the line" effect [Miller et al., 2021]. It explains both observations. v2's augmentation (TrivialAugment's brightness, contrast, colour and sharpness operations, plus grayscale) covered exactly these directions and produced the big gain. Once they were covered, better in-distribution accuracy translated only weakly into test accuracy.

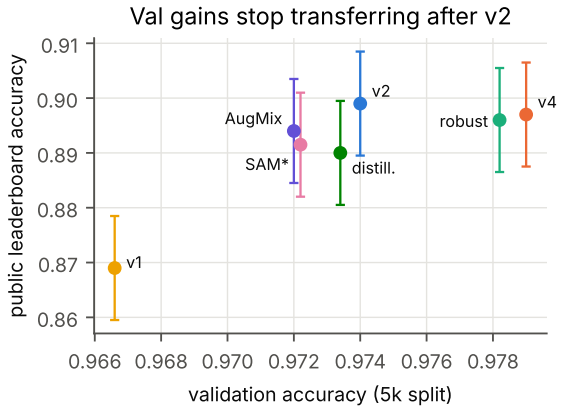


Figure 1: Validation vs. public accuracy. Error bars: ±1 binomial s.e. of a 1,000-image score (±0.95 pt). After v2 every point sits within noise of 89.5%.

Table 3: Training vs. hidden test images (pixel statistics only; test labels never used).

Statistic	Train	Test	Ratio
Mean intensity	0.473	0.491	1.04
Per-image contrast (std)	0.211	0.220	1.04
Edge energy (mean $\lvert\partial_x\rvert$)	0.054	0.059	1.08
Grayscale images	1.1%	3.3%	2.9

A rules question we had to answer. Looking at unlabeled test-image statistics uses no labels, but choosing augmentation from them is close to using the evaluation set for model selection, which the rules forbid. One of our choices, RandomGrayscale($p = 0.05$), was motivated this way. To make sure our results do not depend on it, Michael re-trained v2 on all 50k images *without* it (run 6): it scored 89.8%, the same as v2's 89.9%. Every other augmentation we use is a standard, untuned method.

3.2 We mistook leaderboard noise for regressions

After v2 we trained the same recipe on all 50k images and, separately, with SAM. Both scored lower on the public leaderboard (89.0 and 89.3), and we initially read that as "both did worse". That conclusion was not supported by the data. A score on 1,000 images has a binomial standard error of $\sqrt{0.9 \cdot 0.1/1000} \approx 0.95$ points. A sharper estimate comes from comparing our submissions with each other: two of our models disagree on a median of 6.2% of the test images. If two models are equally accurate, each disagreement is roughly a coin flip, so their public scores differ with standard deviation $\approx \sqrt{d/1000} = 0.8$ points. The spread of our post-v2 scores (89.0–89.9) is about what this predicts.

The full-data model cannot plausibly be worse than v2: it uses the identical recipe with 10% more data. From this point on we compared models on validation accuracy, and on a robustness scorecard we built for the purpose (Section 3.5), and used the leaderboard only as a sanity check.

3.3 SAM: an unequal comparison

SAM reached 97.22% after 200 epochs, against v2's 97.40% after 300. Since SAM costs two forward and backward passes per step, we gave it fewer epochs to keep the wall-clock time comparable. That makes the comparison confounded: we cannot tell whether SAM hurt or simply needed the full schedule. With a fixed GPU quota (30 hours per week on Kaggle) we did not rerun it, and we do not claim SAM fails here.

3.4 Distillation: teachers that were too similar

Distillation is explicitly allowed when the teacher is trained from scratch on the permitted data, so we distilled our own models into a v2-sized student, with temperature 2, a 0.9 weight on the distillation term, and the teacher and student seeing the same augmented batch [Beyer et al., 2022]. On validation the student (97.34%) did not beat its best teacher (v2, 97.40%). The diagnosis is that the ensemble of v2 and SAM scored only 97.52%, just 0.12 points above v2, because two models with the same architecture and augmentation make largely the same errors and so give the student little new information. Michael found that pairing v2 with the AugMix model, which was trained differently, raised the ensemble to 97.68% (+0.28), confirming that diversity is what matters. Our teachers were also trained with label smoothing, which is known to make worse teachers [Müller et al., 2019].

We also considered a large teacher. The rules allow a teacher of any size (only the final model is capped), so we implemented a WRN-28-10 [Zagoruyko and Komodakis, 2016] with 36.5M parameters, with a guard that refuses to write predictions for any model over 5M. We dropped it before training once v4 reached 97.90%: the published result for a WRN-28-10 trained with TrivialAugment is about 97.5% [Müller and Hutter, 2021], so at our budget the teacher was unlikely to beat the student, and training it would have used about half of our remaining GPU quota.

3.5 Robustness gains in the wrong directions

Because the leaderboard could not measure small improvements, we built a robustness scorecard: validation accuracy on seven deterministic shifts of our own validation images. The shifts are brightness +15%, contrast +25%, sharpening, blur, grayscale, JPEG at quality 60 and Gaussian noise, inspired by CIFAR-10-C [Hendrycks and Dietterich, 2019]. JPEG and noise never appear in training. Our robust recipe replaced TrivialAugment with a random choice between TrivialAugment and AugMix and added per-image colour jitter, grayscale and blur, which we moved to the GPU because AugMix alone made the CPU data loader too slow to keep up with the GPU. It raised mean shifted accuracy from 94.27% to 94.87% (+2.3 points on the held-out JPEG shift, +1.8 on blur) at almost no cost in clean accuracy (Figure 2). It did not raise the public score (89.6 vs. 89.7). The scorecard explains why: on the directions in which the test set actually differs (brightness, contrast, sharpness and grayscale), v4 and the robust model are already within 0.2 points of each other. Michael's earlier AugMix run showed the same pattern: +3.2 points on his own 7-corruption benchmark, but 89.4% on the leaderboard.

3.6 Fine-tuning from a converged model

Our last experiment continued training the v4 validation checkpoint for 200 more epochs, replacing two thirds of the images in each batch with the harder of two random photometric views, in the spirit of AugMax [Wang et al., 2021]. Restarting at learning rate 0.018 knocked clean accuracy from 97.90% to 84.3% within two epochs, and the run only recovered to 97.44% (10-view: 97.86% $\rightarrow$ 97.48%). A converged model sits in a narrow region of weight space; a fresh high-learning-rate phase undoes most of what the cosine schedule achieved, and 200 epochs were not enough to recover.

3.7 Engineering mistakes

- **A false alarm from float32 arithmetic.** Our first comparison of train and test statistics showed a training-set mean of 0.328 in every channel, far from CIFAR-10's 0.49/0.48/0.45, which suggested the training set had been darkened. The cause was float32 accumulation error over 1.5×10^8 values; recomputing in float64 gave the expected values. Since then all statistics, including the normalisation constants, are computed in float64.
- **Path assumptions.** Our first Kaggle run failed because the competition data was mounted one directory deeper than expected; the script now searches for `train_labels.csv`.
- **A race condition.** Running two jobs in one session, both processes wrote the same image-cache file at once. We caught this in a local test, before it cost GPU time, and fixed it with per-process temporary files and an atomic rename.

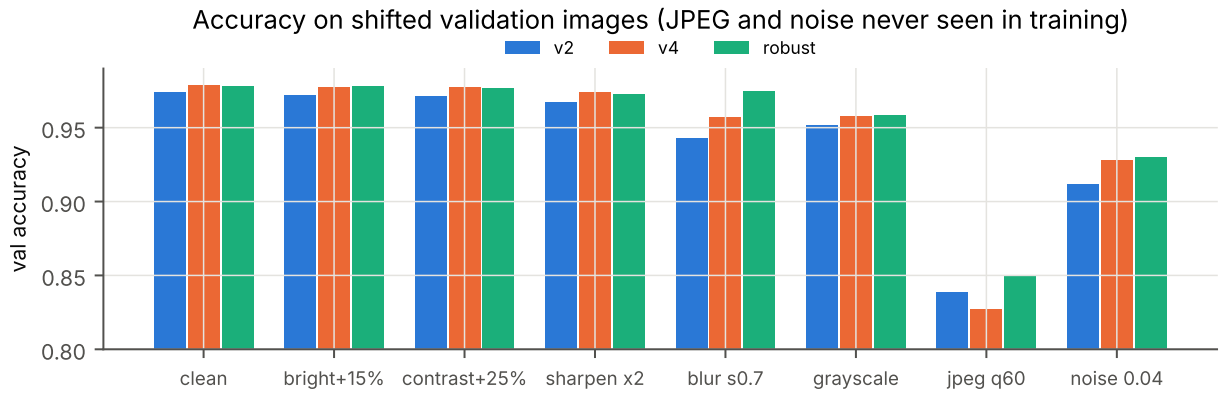


Figure 2: Robustness scorecard on shifted copies of our 5k validation images (EMA weights, flip-averaged). The robust recipe gains most on blur and JPEG, which do not resemble the test shift; on brightness, contrast, sharpening and grayscale it is level with v4.

- **Naming.** Our code calls v2 "ResNet-26"; by the standard convention it has 24 layers, and the final model 26. This report uses the corrected names.

4 Results

Final model. SE-ResNet-26 (Table 1): stem conv3×3 with 64 channels; stage 1 has five blocks at 64 channels and 32×32; stage 2 has four blocks at 128 channels and 16×16; stage 3 has three blocks at 256 channels and 8×8. Each block has an SE gate with reduction 16, and the network ends with global average pooling and a 256→10 linear layer. It has **4,810,754 trainable parameters** (68.7% in stage 3, 23.4% in stage 2, 7.8% in stage 1; the SE gates account for 39,608 or 0.8%). We submitted two variants, both trained on all 50k images for 600 epochs: `final` (v4 recipe) and `final^2` (robust recipe).

Table 4: Final results. Validation numbers come from the 45k twin of each submission (same recipe and seed). Private scores were released after the deadline and decide the ranking.

Submission	Params	Val (flip)	Val (10-view)	Shifted val	Public	Private
`final` (v4)	4,810,754	97.90%	97.86%	94.27%	89.7%	TBD
`final^2` (robust)	4,810,754	97.82%	97.92%	94.87%	89.6%	TBD
v2 (best public score)	4,697,162	97.40%	97.54%	93.65%	89.9%	TBD

Figure 3 shows training curves. With heavy augmentation, validation accuracy keeps climbing as the learning rate decays: the last 20% of the schedule still adds 1.2–1.4 points (v4: 96.6% → 97.9%; v2: 96.0% → 97.4%), consistent with longer schedules helping. v1, with weak augmentation, shows the clearest overfitting: 99.45% training accuracy against 96.66% on validation.

Reproducibility. The repository contains the exact script and executed Kaggle notebook for every run, the EMA checkpoints of both submissions and their validation twins, and `results.ipynb`. That notebook prints the parameter counts, recomputes validation accuracy from the checkpoints (it reproduces the logged values exactly: 97.40%, 97.90% and 97.82% for v2, v4 and robust), and regenerates both submission files (the regenerated predictions agree with the uploaded files on 2,000 of 2,000 test images for `final` and 1,999 of 2,000 for `final^2`).

5 Lessons and limitations

- **Measure the noise before chasing a leaderboard.** After v2, almost every change we made moved the public score by less than its resolution, and we read two of those moves as regressions. Computing the standard error and the disagreement between submissions first would have told us to rely on validation sooner.
- **Validation accuracy and test accuracy diverge under distribution shift.** Augmentation that covers the directions of the shift helped a lot; generic robustness and extra in-distribution accuracy helped little.

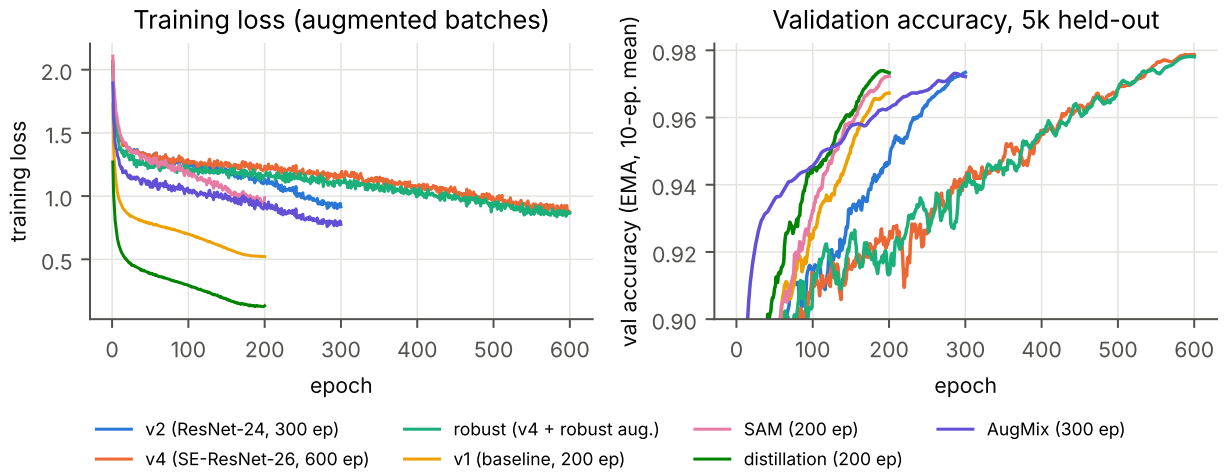


Figure 3: Training loss and validation accuracy (EMA) for every run with a validation split. Loss levels are not comparable across recipes, because CutMix, label smoothing and distillation change the targets.

- **Diversity beats similarity in distillation and ensembling.** Same-recipe teachers added almost nothing.
- **Compare at equal budgets.** Our SAM comparison was confounded by its shorter schedule.
- **Limitations.** One seed per configuration; no confidence intervals on validation differences below about 0.2 points (the binomial s.e. on 5,000 images); several changes were made together (v1→v2), so their individual contributions are not separated.

Contributions. Uvindu Salgado built the training pipeline (v1–v6), the distillation, SE and robustness experiments, the robustness scorecard and the analysis. Michael Wang ran the AugMix experiment with its corruption benchmark, the v2 re-run on 50k images without grayscale, and the teacher-diversity analysis.

Acknowledgements. Our residual block, CIFAR-style stem and baseline optimizer settings (SGD, learning rate 0.1, momentum 0.9, weight decay 5×10^{-4}, cosine schedule, crop and flip augmentation) follow the conventions of Liu [2017]. All code in our repository was written independently. The report uses the arxiv-style template [Kour, 2020].

References

Lucas Beyer, Xiaohua Zhai, Amélie Royer, Larisa Markeeva, Rohan Anil, and Alexander Kolesnikov. Knowledge distillation: A good teacher is patient and consistent. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

Terrance DeVries and Graham W. Taylor. Improved regularization of convolutional neural networks with cutout. *arXiv preprint arXiv:1708.04552*, 2017.

Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-aware minimization for efficiently improving generalization. In *International Conference on Learning Representations (ICLR)*, 2021.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations (ICLR)*, 2019.

Dan Hendrycks, Norman Mu, Ekin D. Cubuk, Barret Zoph, Justin Gilmer, and Balaji Lakshminarayanan. AugMix: A simple data processing method to improve robustness and uncertainty. In *International Conference on Learning Representations (ICLR)*, 2020.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, 2015.

George Kour. arxiv-style: LaTeX template for arXiv preprints. https://github.com/kourgeorge/arxiv-style, 2020.

Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

Kuang Liu. pytorch-cifar: Train CIFAR10 with PyTorch. https://github.com/kuangliu/pytorch-cifar, 2017.

Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018.

John Miller, Rohan Taori, Aditi Raghunathan, Shiori Sagawa, Pang Wei Koh, Vaishaal Shankar, Percy Liang, Yair Carmon, and Ludwig Schmidt. Accuracy on the line: On the strong correlation between out-of-distribution and in-distribution generalization. In *International Conference on Machine Learning (ICML)*, 2021.

Rafael Müller, Simon Kornblith, and Geoffrey Hinton. When does label smoothing help? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Samuel G. Müller and Frank Hutter. TrivialAugment: Tuning-free yet state-of-the-art data augmentation. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Boris T. Polyak and Anatoli B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.

Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do CIFAR-10 classifiers generalize to CIFAR-10? *arXiv preprint arXiv:1806.00451*, 2018.

Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do ImageNet classifiers generalize to ImageNet? In *International Conference on Machine Learning (ICML)*, 2019.

Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *International Conference on Machine Learning (ICML)*, 2013.

Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

Aditya Thakur, Harish Chauhan, and Nikunj Gupta. Efficient ResNets: Residual network design. *arXiv preprint arXiv:2306.12100*, 2023.

Haotao Wang, Chaowei Xiao, Jean Kossaifi, Zhiding Yu, Anima Anandkumar, and Zhangyang Wang. AugMax: Adversarial composition of random augmentations for robust training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. CutMix: Regularization strategy to train strong classifiers with localizable features. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. In *British Machine Vision Conference (BMVC)*, 2016.